

TRÌNH BIÊN DỊCH (Giới thiệu)

Phạm Nguyên Khang
BM. Khoa học máy tính

pnkhang@cit.ctu.edu.vn

CẦN THƠ, 12 - 2012

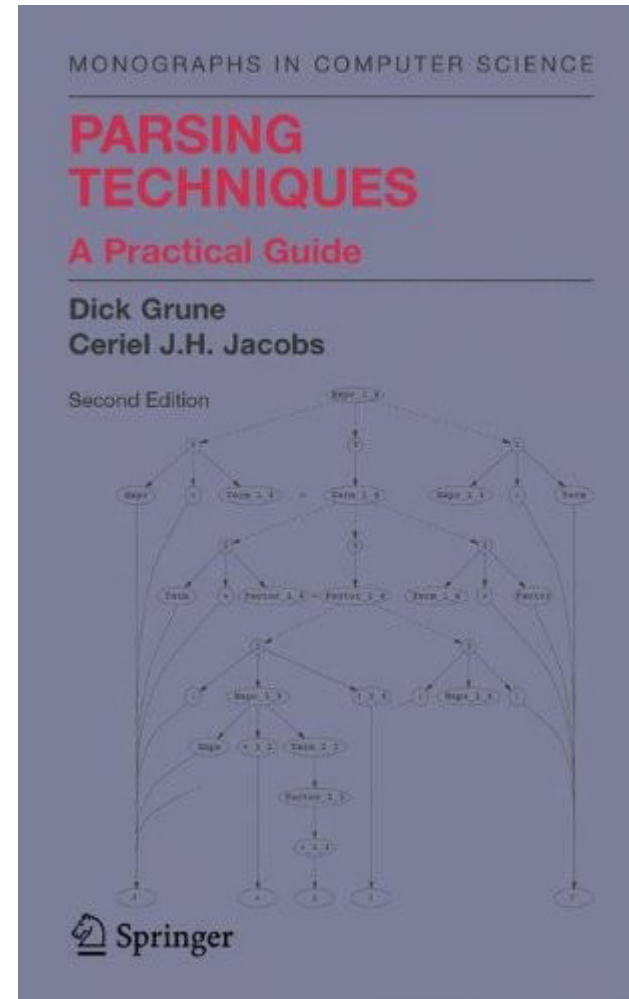
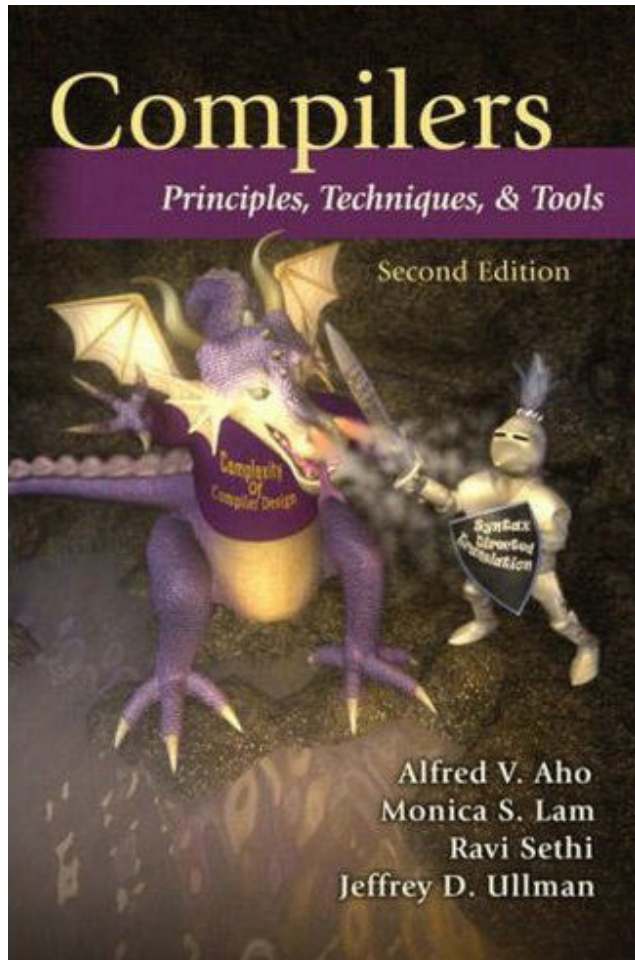
Giới thiệu

- Thông tin môn học
- Tại sao học trình biên dịch
- Lược sử trình biên dịch
- Cấu trúc trình biên dịch

Thông tin môn học

- Giảng viên:
 - Phạm Nguyên Khang (pnkhang@cit.ctu.edu.vn)
- Hướng dẫn thực hành:
 - Võ Trí Thức (vtthuc@cit.ctu.edu.vn)
 - Nguyễn Bá Diệp (nbdiep@cit.ctu.edu.vn)

Tài liệu tham khảo



Tài liệu tham khảo

- Phan Thị Tươi, Trình Biên Dịch, Trường Đại học kỹ thuật Tp.HCM, NXB Giáo dục, 1998.
- Nguyễn Văn Linh, Võ huỳnh Trâm, Phan Phương Lan, Trình biên dịch, Khoa CNTT&TT - ĐHCT, 2003
- Phạm Nguyên Khang, Bài giảng Trình Biên Dịch (slides), Khoa CNTT&TT - ĐHCT

Phương pháp giảng dạy

- Lý thuyết:
 - Thuyết trình + Minh họa bằng chương trình máy tính
 - Đặt vấn đề + thảo luận
 - Làm bài tập nhóm
- Thực hành:
 - Bài thực hành có lời giải (ví dụ)
 - Bài thực hành có hướng dẫn (bắt buộc)
 - Bài thực hành nâng cao (điểm thưởng)

Đánh giá môn học

- Kiểm tra giữa kỳ (30%)
- Bài tập lập trình (lập trình + báo cáo), làm theo nhóm: 30%
- Thi viết (40%)

Tại sao học trình biên dịch ?

- Xây dựng **hệ thống phần mềm lớn, nhiều chức năng**
- Thấy được ứng dụng của lý thuyết vào thực tế
- Học cách **xây dựng ngôn ngữ lập trình (programming languages)**
- Học **cách thức hoạt động của ngôn ngữ lập trình**
- Học **cách dung hòa (tradeoffs) trong thiết kế ngôn ngữ lập trình (language design)**

Lược sử trình biên dịch

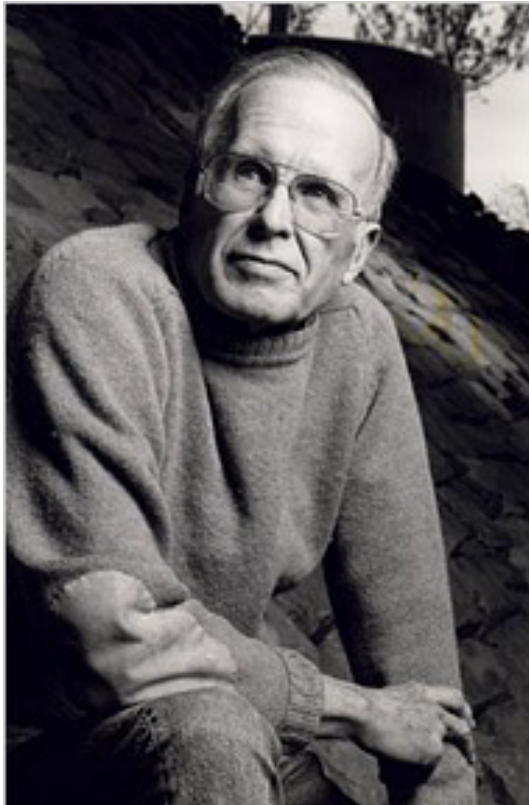
- Đầu tiên, không có gì cả
- Kể đến, mã máy (machine code)
- Tiếp sau đó, hợp ngữ (assembly languages)
- Lập trình trở nên có giá trị; 50% giá thành của máy tính là công của lập trình

Ngôn ngữ lập trình cấp cao



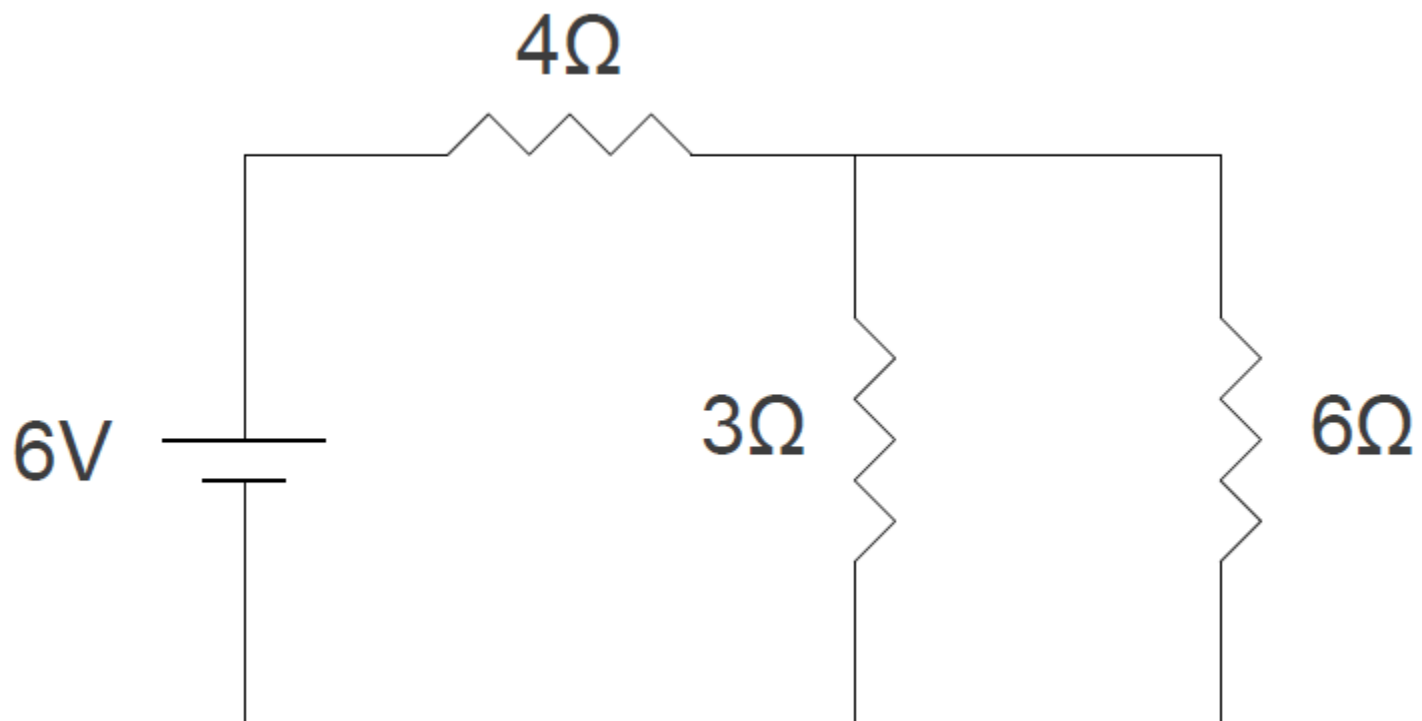
- Chuẩn đô đốc/Đề đốc (Rear Admiral) Grace Murray Hopper
- Người phát minh ra ngôn ngữ lập trình **A-0**, ngôn ngữ lập trình **COBOL** và thuật ngữ **compiler** (trình biên dịch)

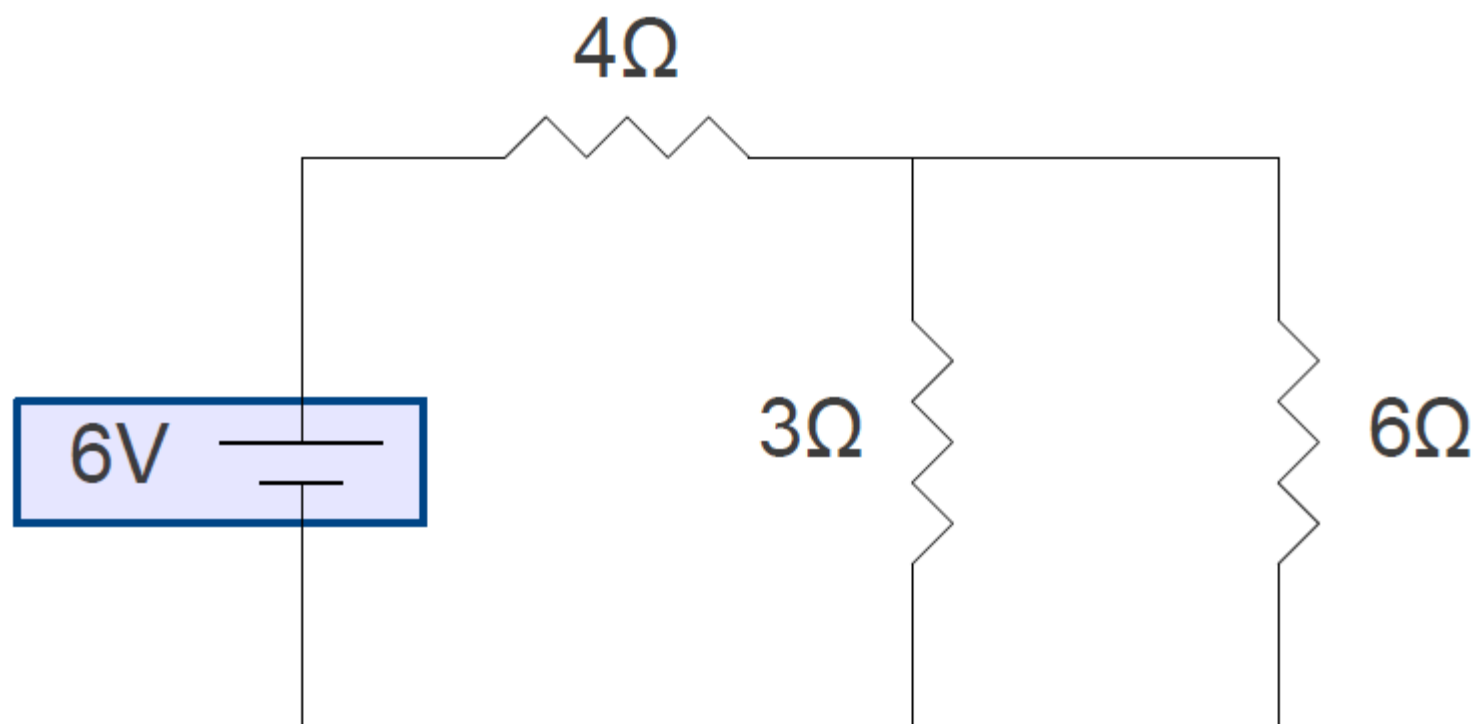
Ngôn ngữ lập trình cấp cao

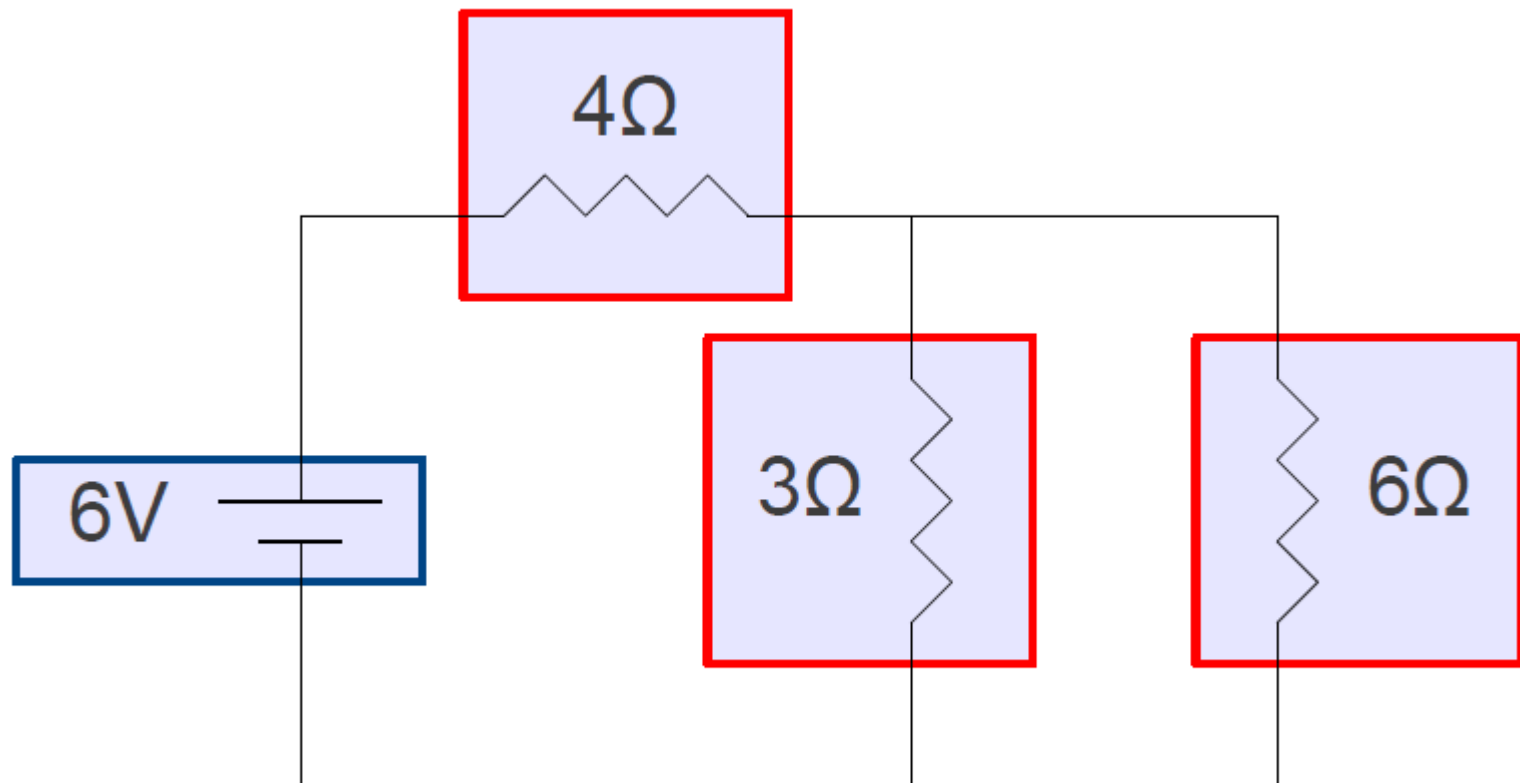


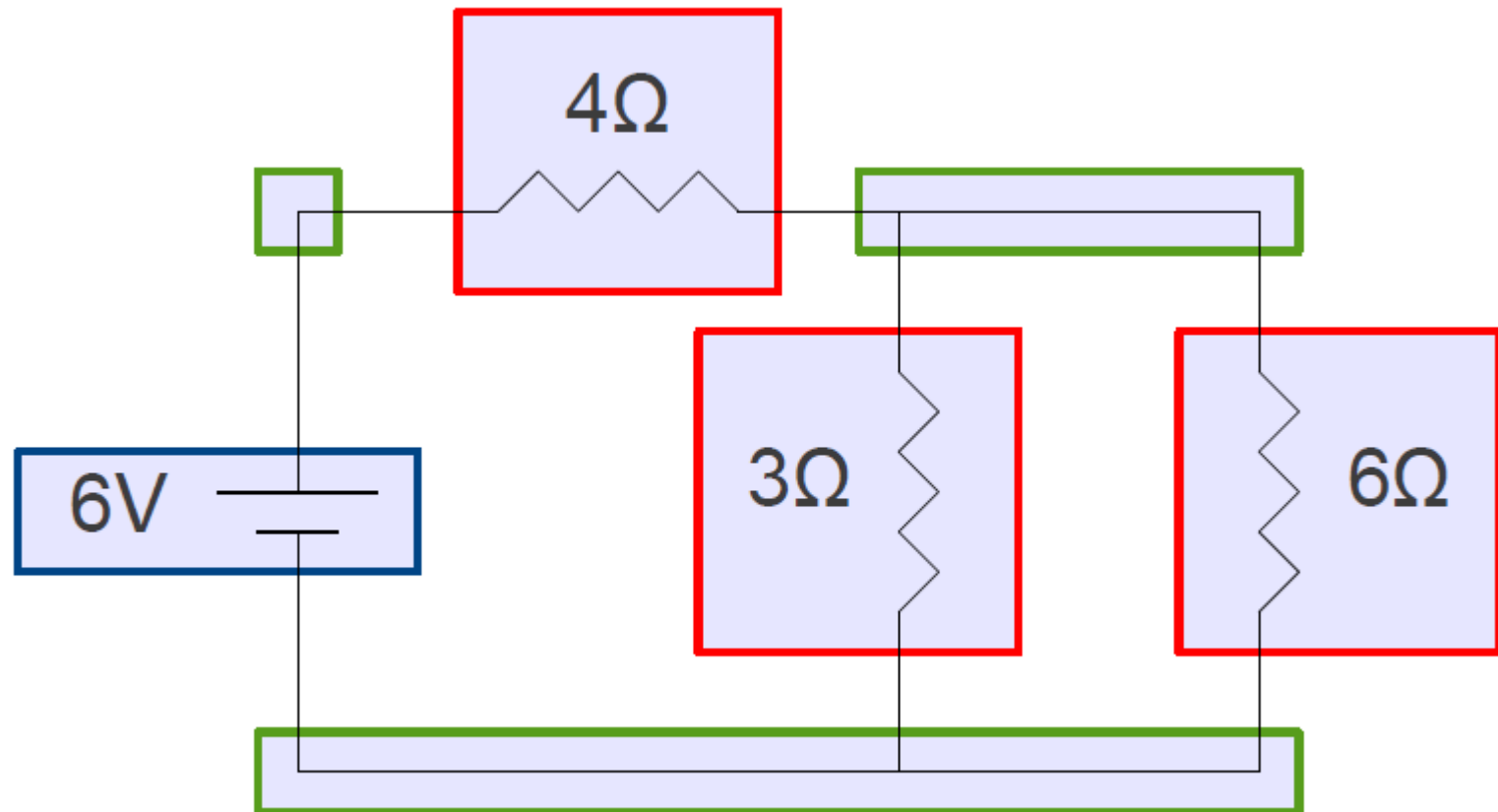
- John Backus
- Trưởng nhóm phát minh ra ngôn ngữ lập trình FORTRAN.
- Người phát minh ra **dạng chuẩn Backus-Naur (BNF)** sử dụng trong ngôn ngữ hình thức

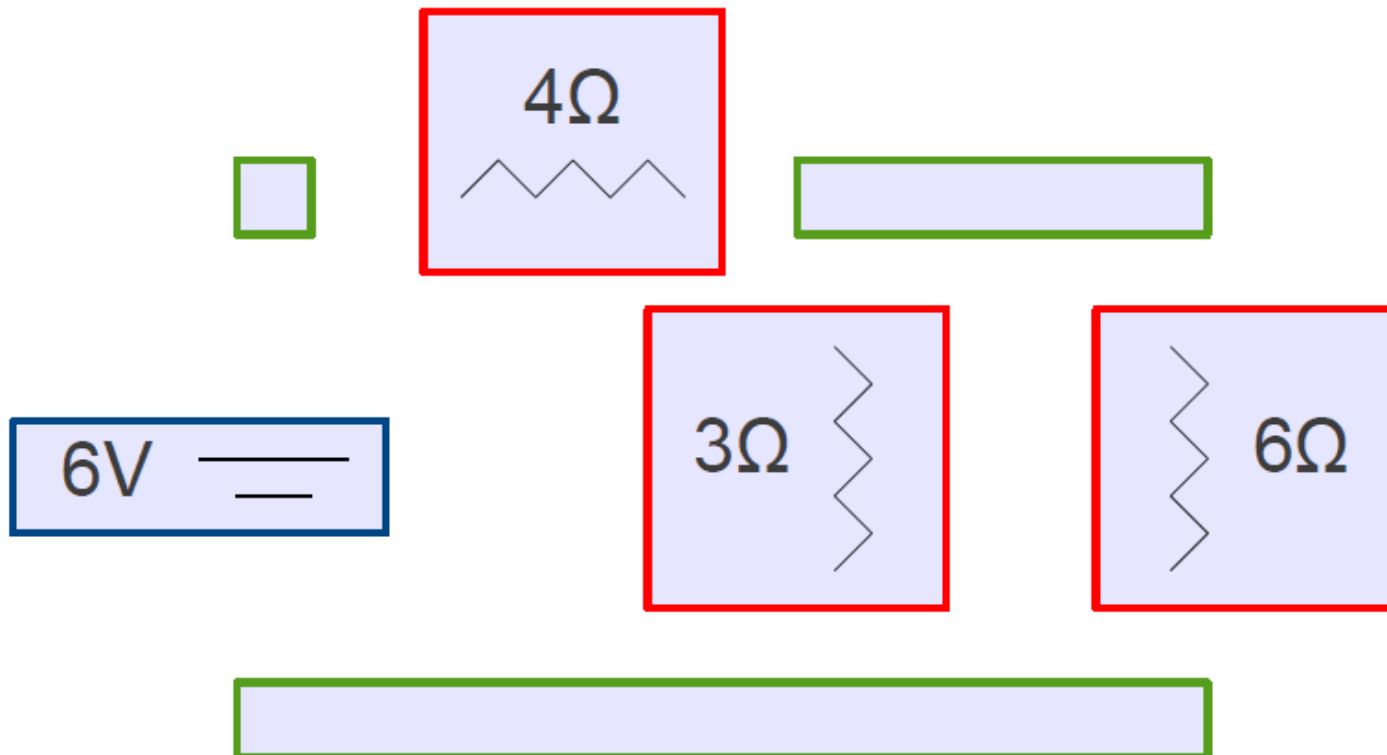
Trình biên dịch hoạt động như thế nào ?

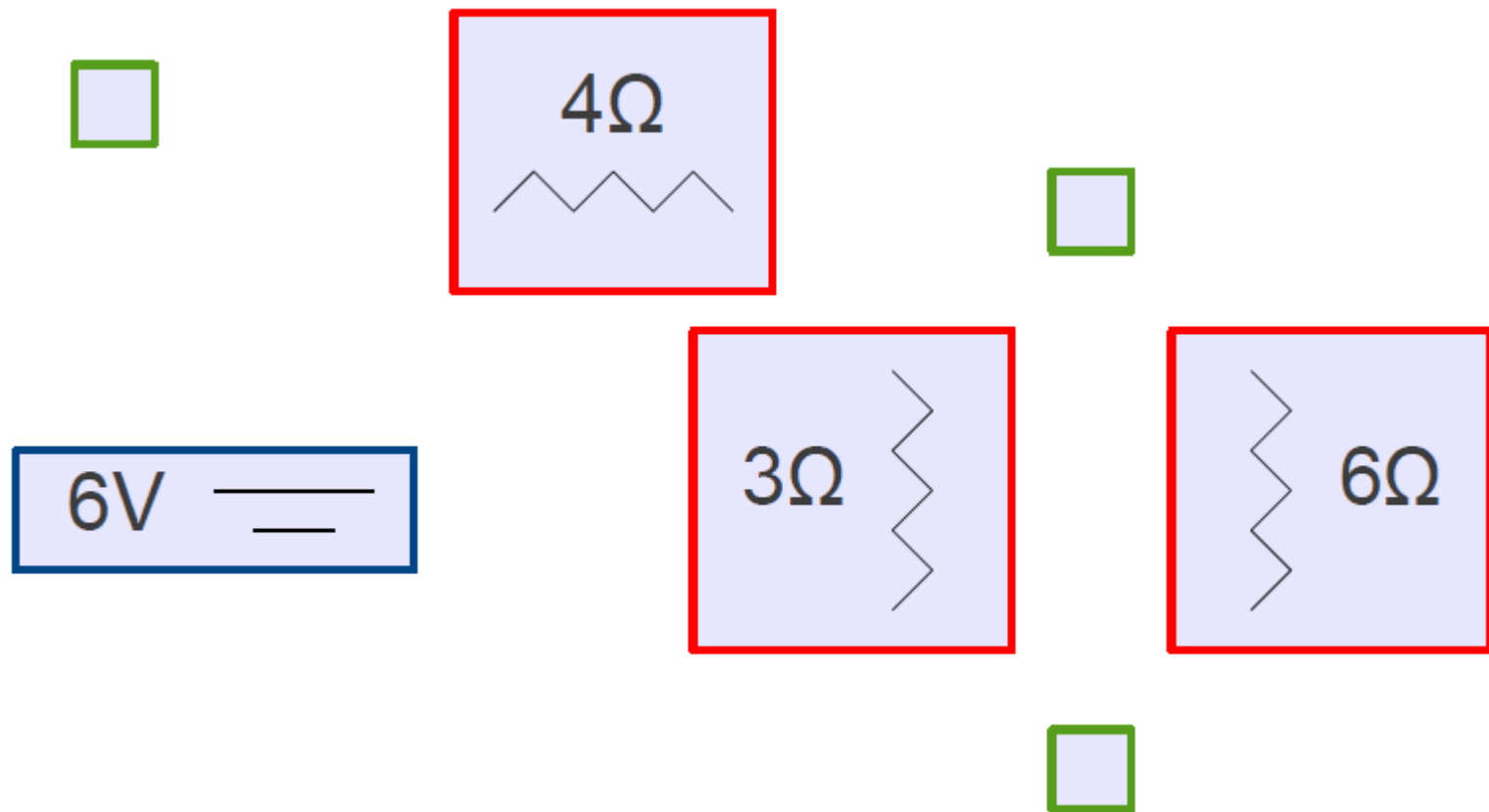


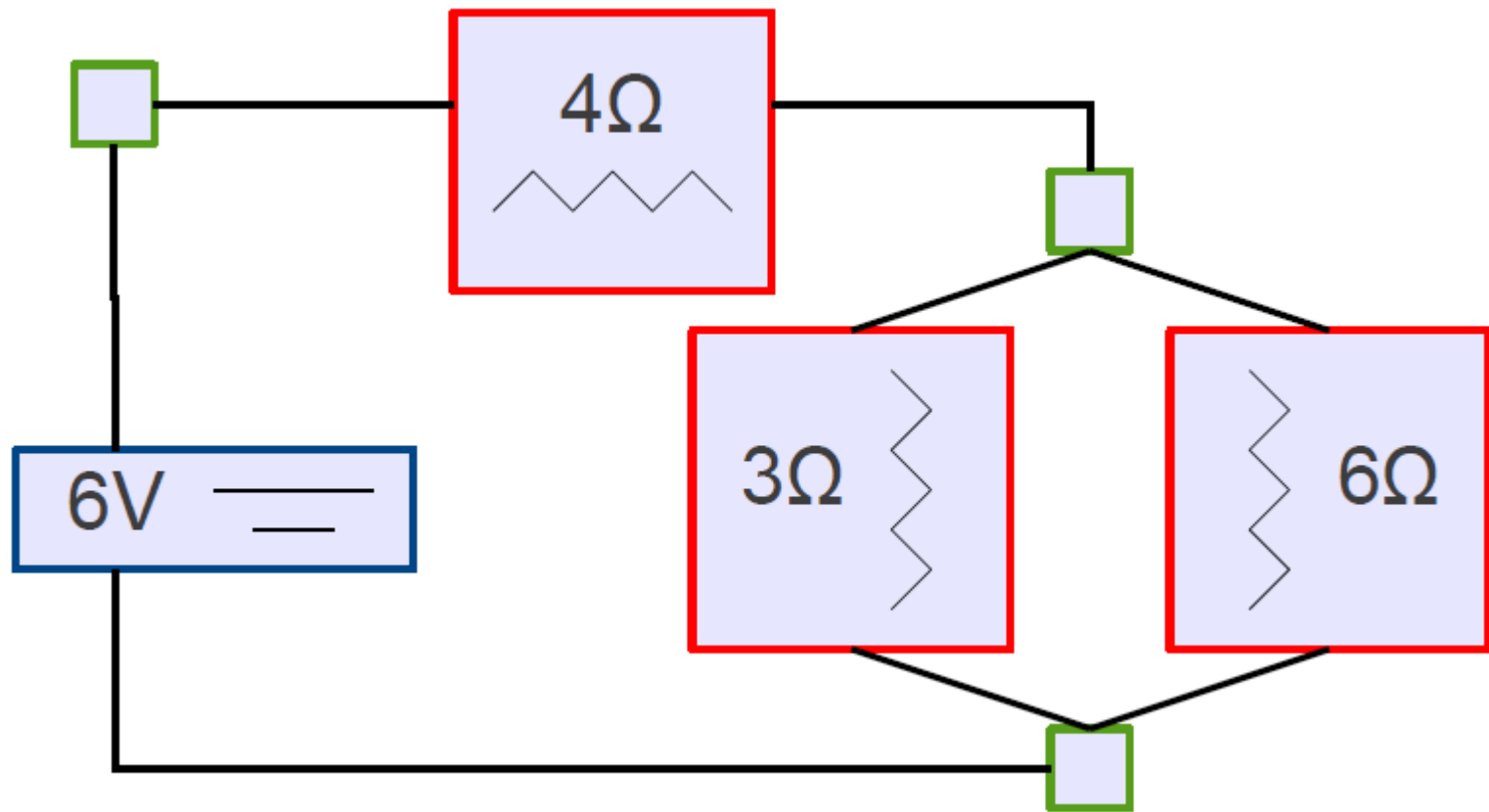


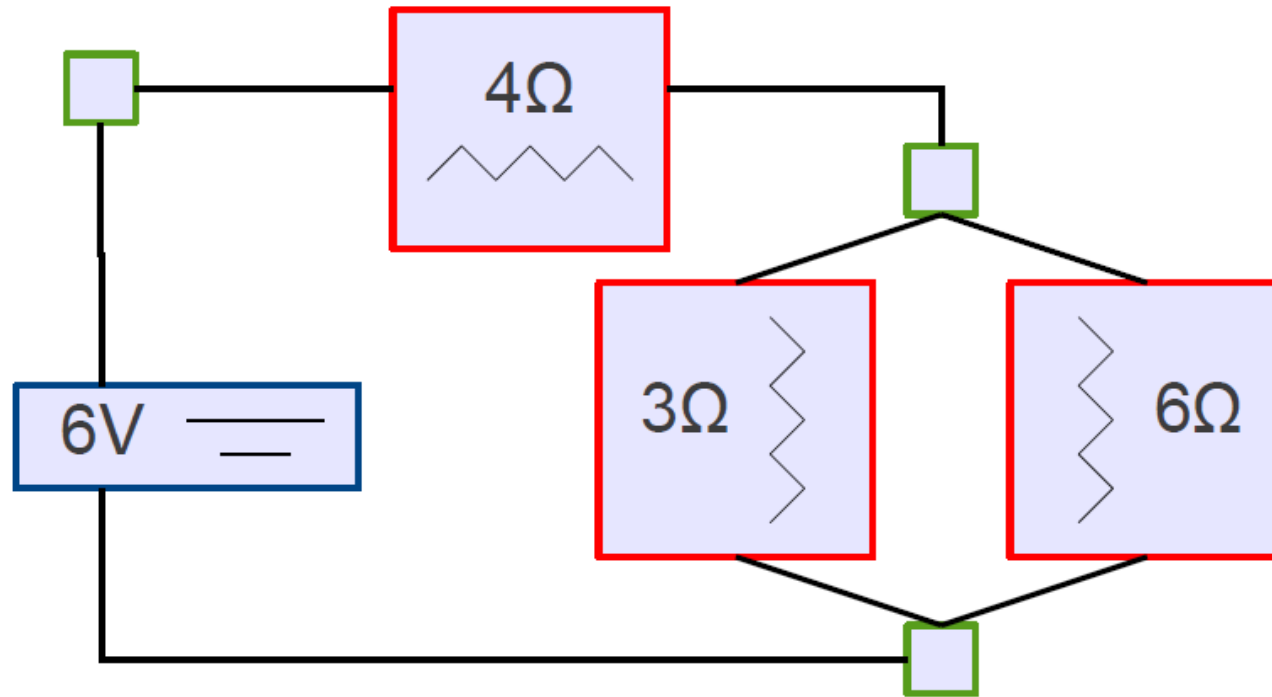




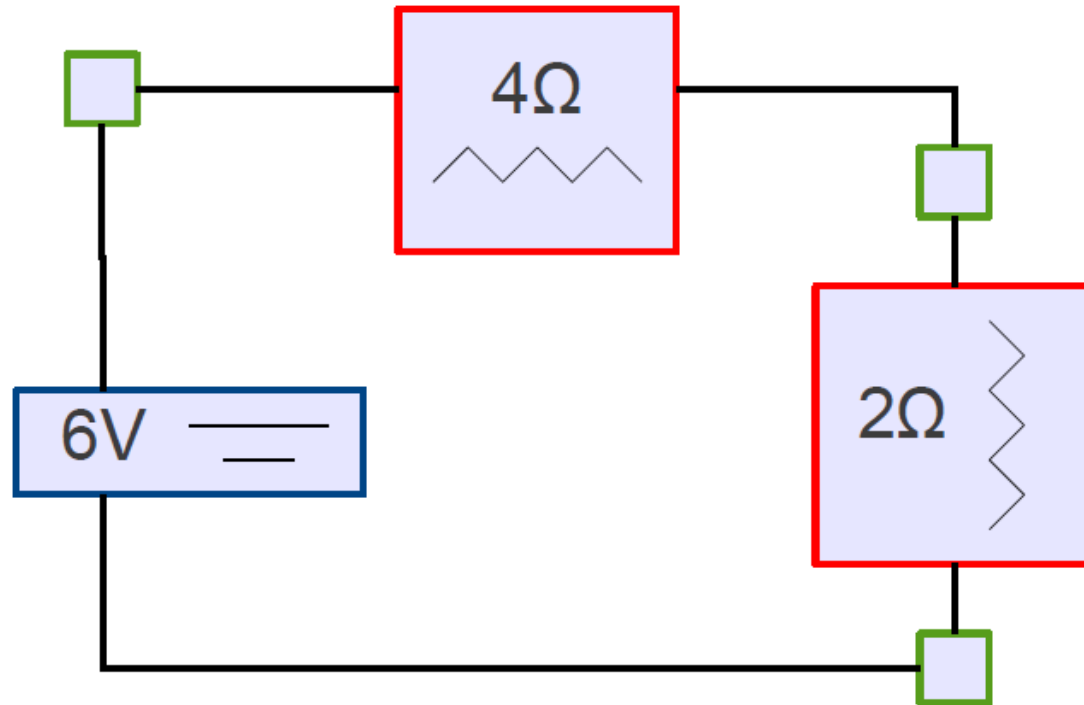




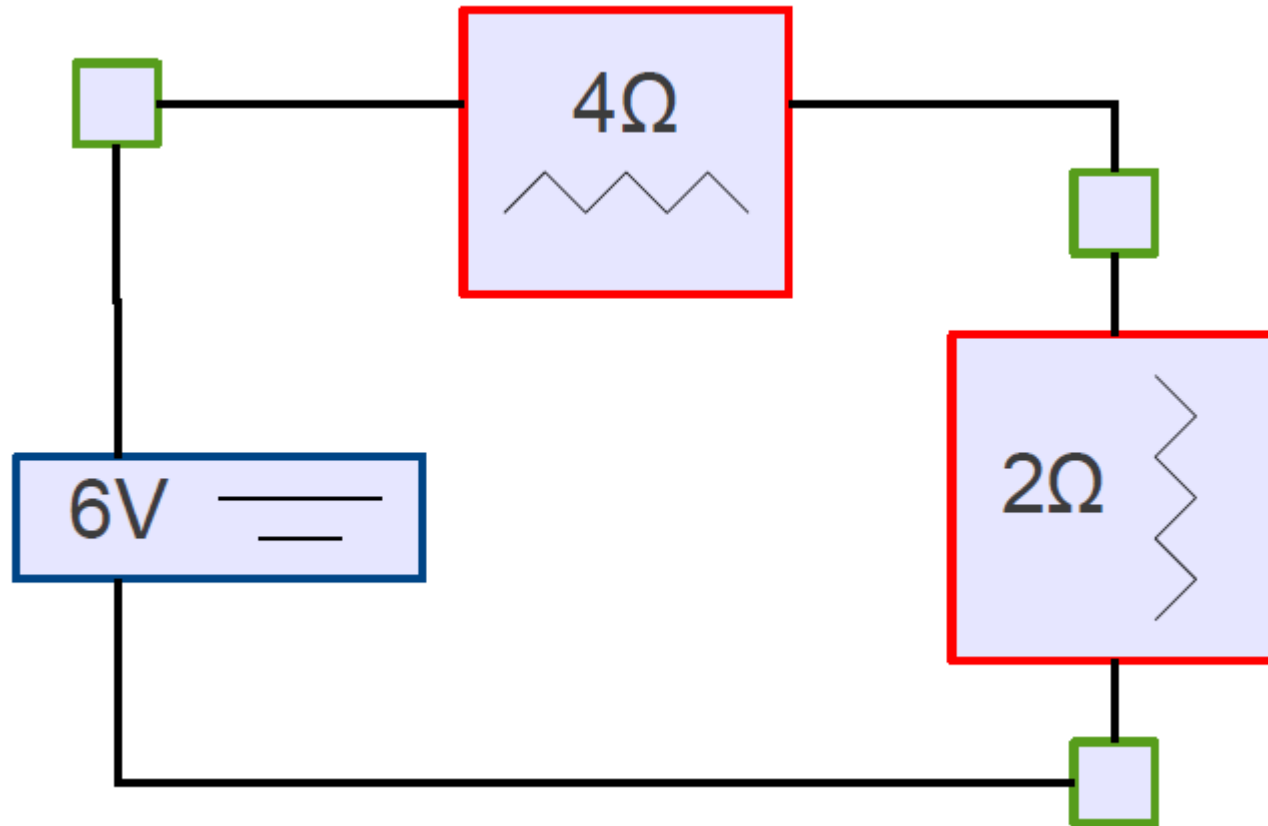


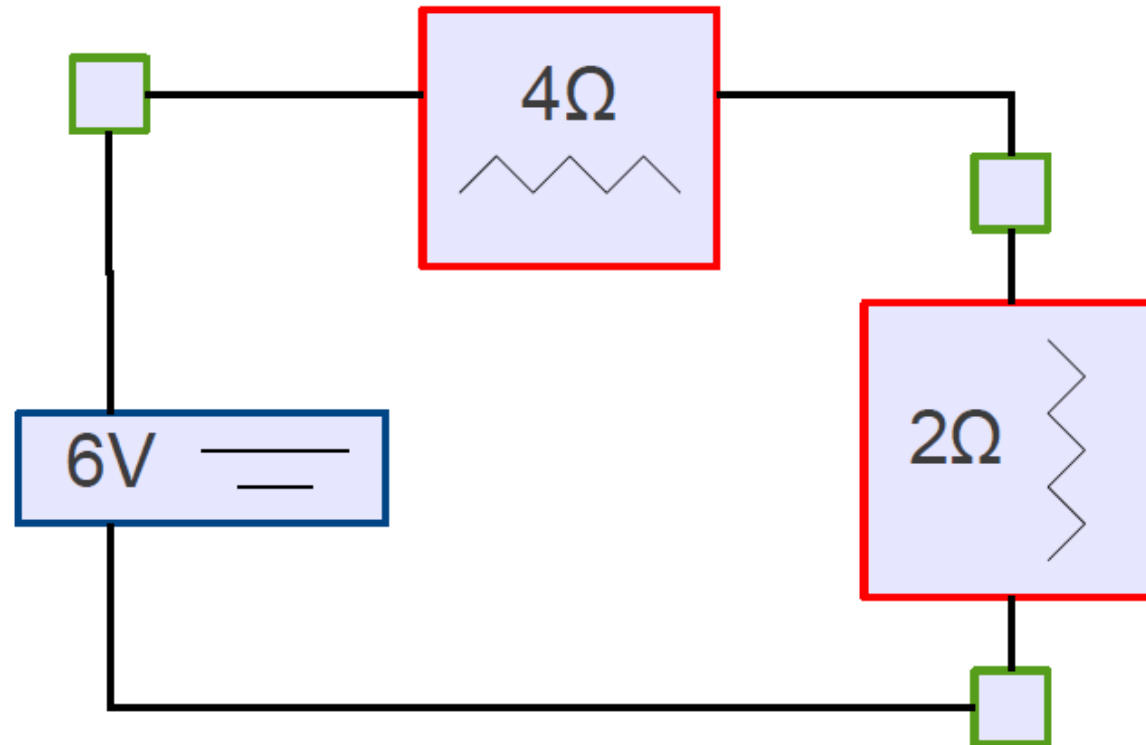


$$\frac{1}{\frac{1}{3\Omega} + \frac{1}{6\Omega}} = 2\Omega$$

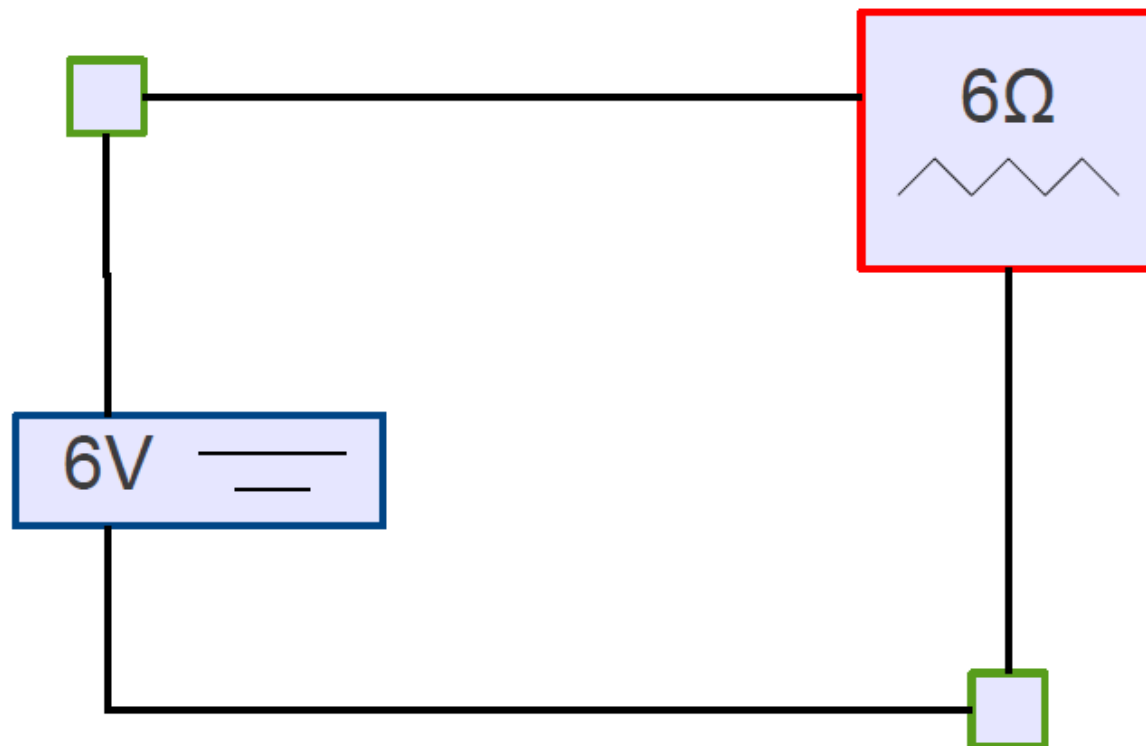


$$\frac{1}{\frac{1}{3\Omega} + \frac{1}{6\Omega}} = 2\Omega$$

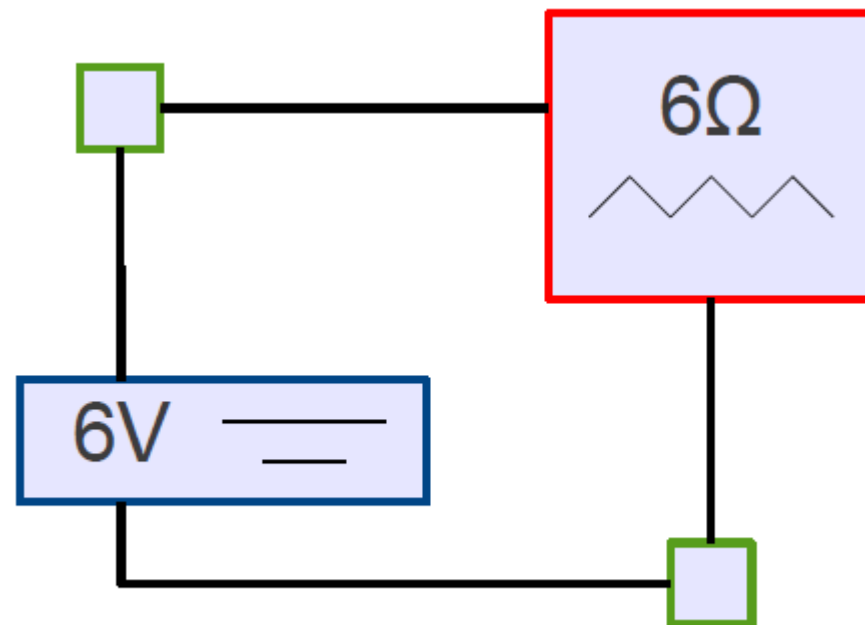


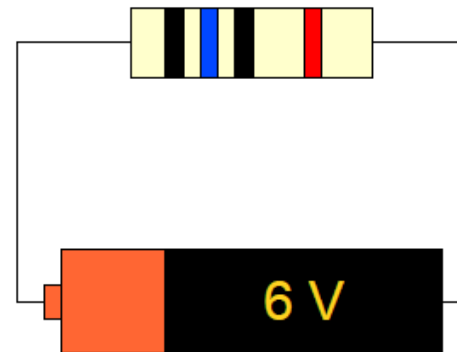
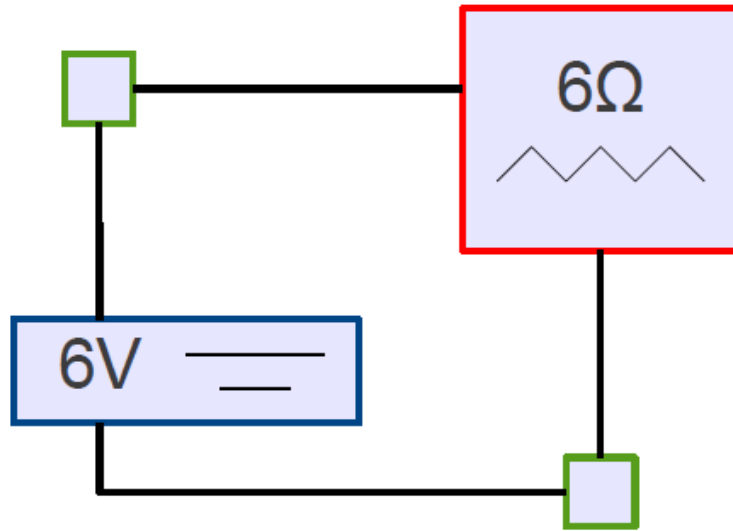


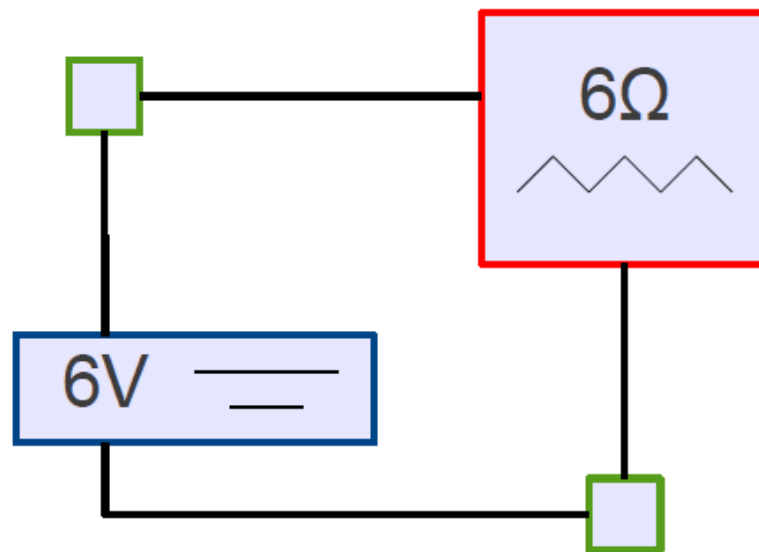
$$4\Omega + 2\Omega = 6\Omega$$



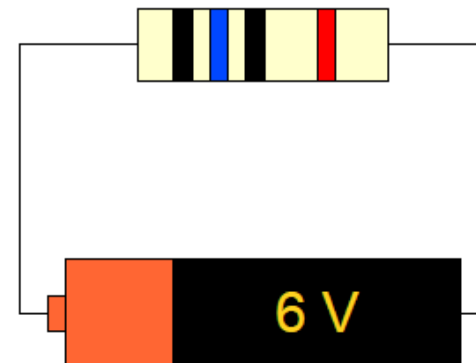
$$4\Omega + 2\Omega = 6\Omega$$

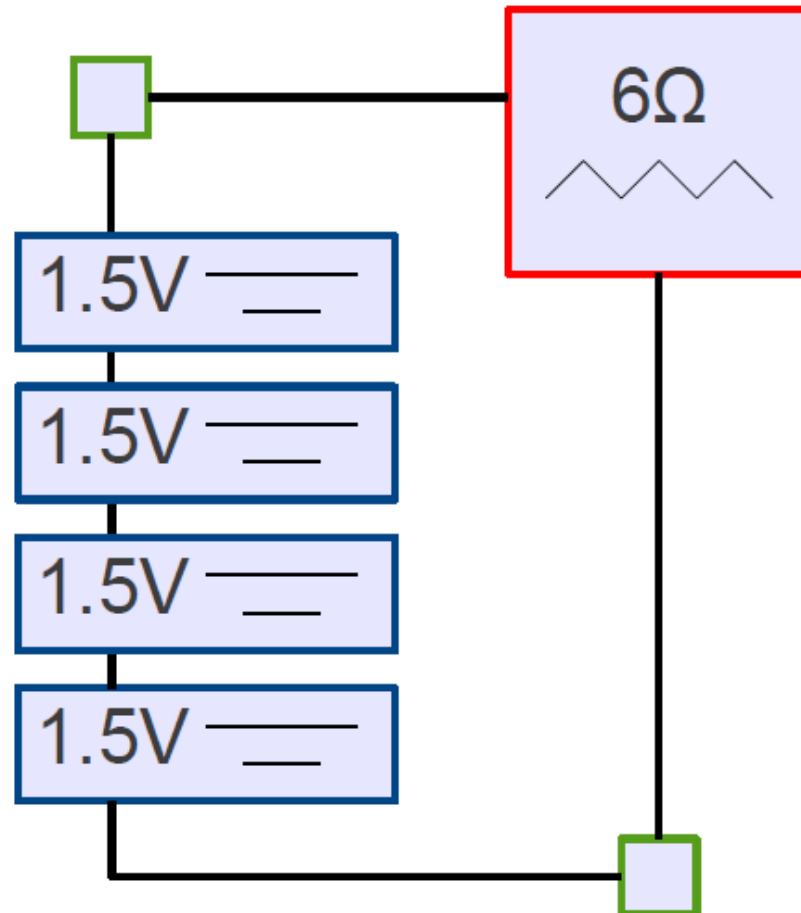


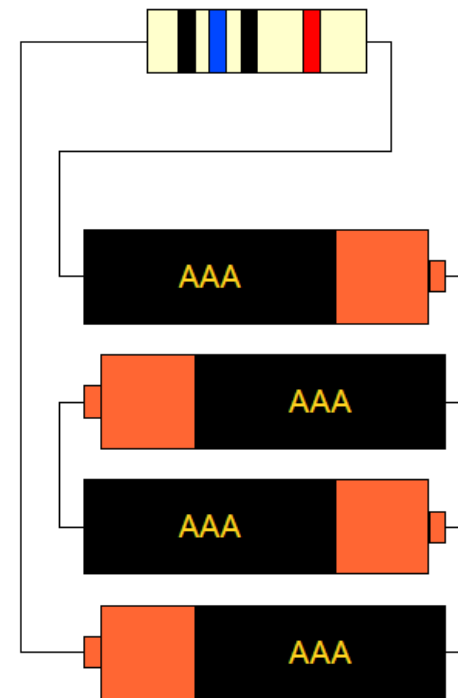
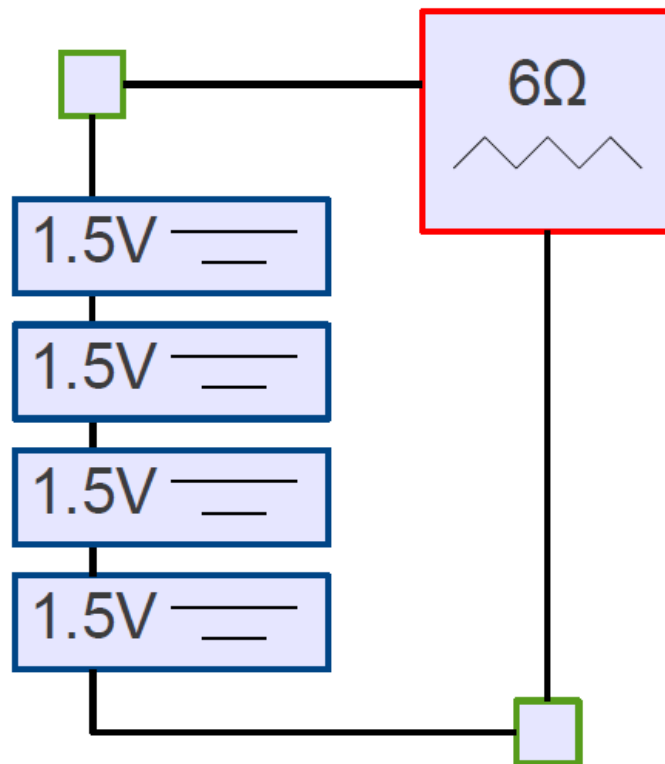


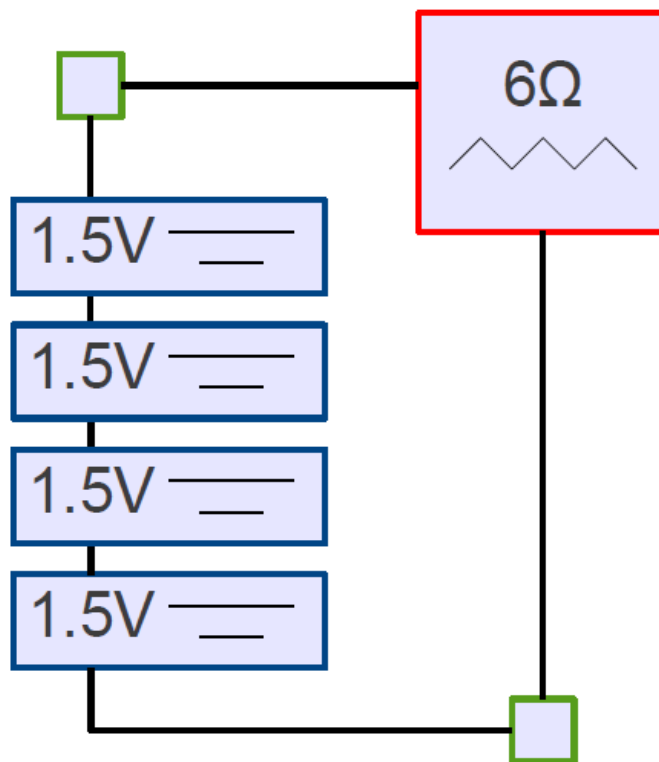


Total Cost: \$4.75

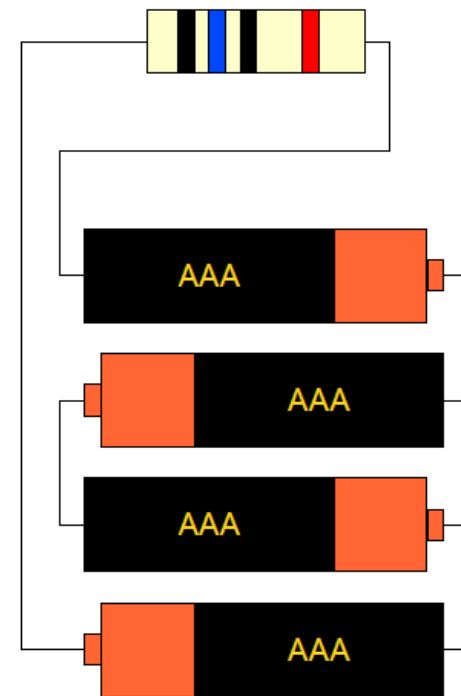








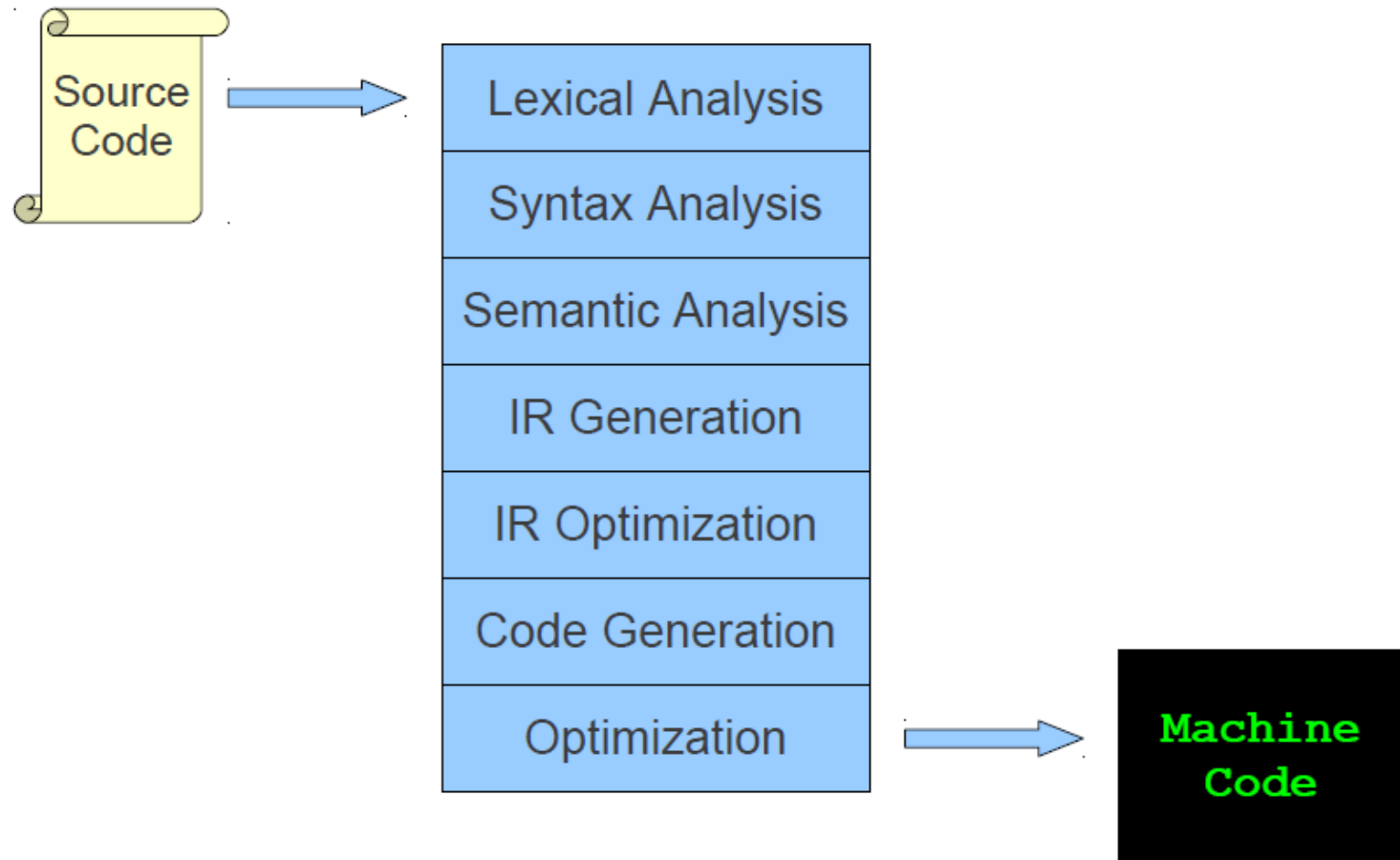
Total Cost: \$1.00



Từ mô tả đến cài đặt

- Phân tích từ vựng (Lexical analysis): xác định **các thành phần logic** của bản mô tả (ví dụ: chương trình nguồn).
- Phân tích cú pháp (syntax analysis): xác định **mối quan hệ** giữa các thành phần.
- Phân tích ngữ nghĩa (semantic analysis): xác định **ngữ nghĩa của toàn bộ cấu trúc**.
- Sinh mã trung gian (IR Generation): thiết kế một cấu trúc khả dĩ (possible structure)
- Tối ưu mã trung gian (IR Optimization): đơn giản hóa cấu trúc
- Sinh mã (Generation): Sinh cấu trúc
- Tối ưu mã (Optimization): Cải tiến cấu trúc sinh ra

Cấu trúc một trình biên dịch hiện đại



```
while (y < z) {  
    int x = a + b;  
    y += x;  
}
```

Lexical Analysis

Syntax Analysis

Semantic Analysis

IR Generation

IR Optimization

Code Generation

Optimization

```

while (y < z) {
    int x = a + b;
    y += x;
}

```

```

T_While
T_LeftParen
T_Identifier y
T_Less
T_Identifier z
T_RightParen
T_OpenBrace
T_Int
T_Identifier x
T_Assign
T_Identifier a
T_Plus
T_Identifier b
T_Semicolon
T_Identifier y
T_PlusAssign
T_Identifier x
T_Semicolon
T_CloseBrace

```

Lexical Analysis

Syntax Analysis

Semantic Analysis

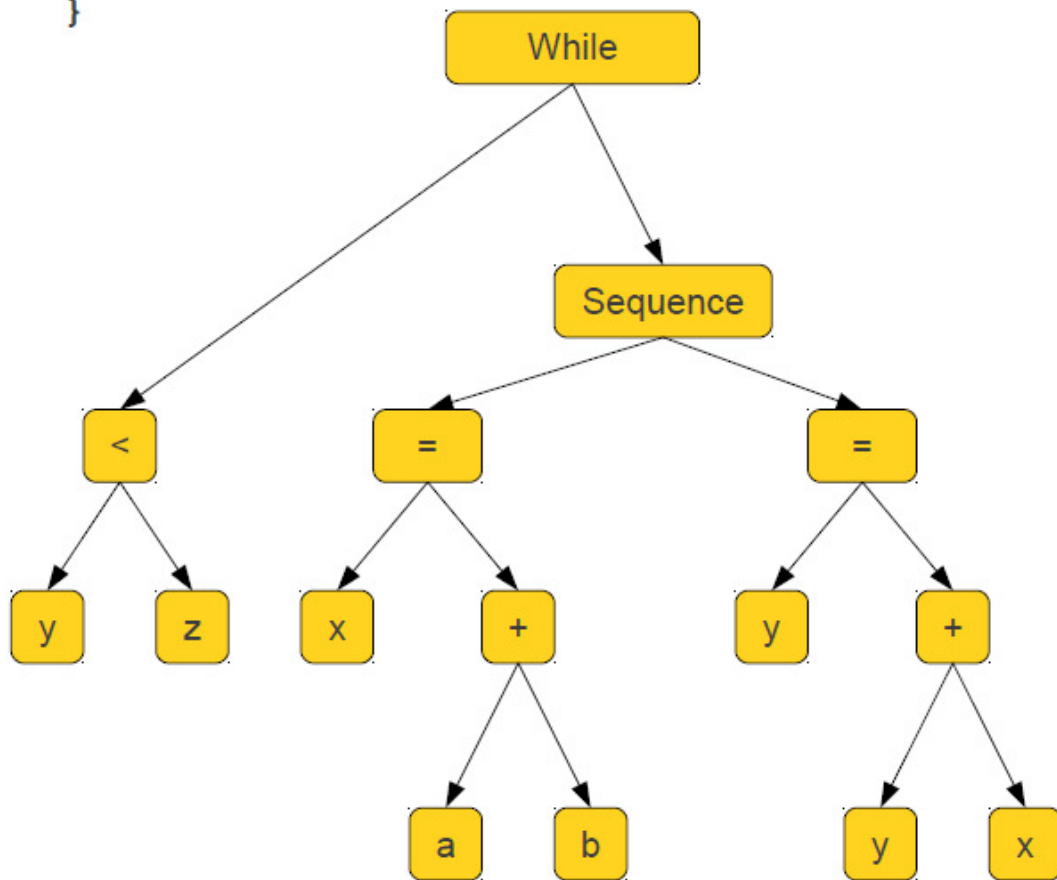
IR Generation

IR Optimization

Code Generation

Optimization

```
while (y < z) {  
    int x = a + b;  
    y += x;  
}
```



Lexical Analysis

Syntax Analysis

Semantic Analysis

IR Generation

IR Optimization

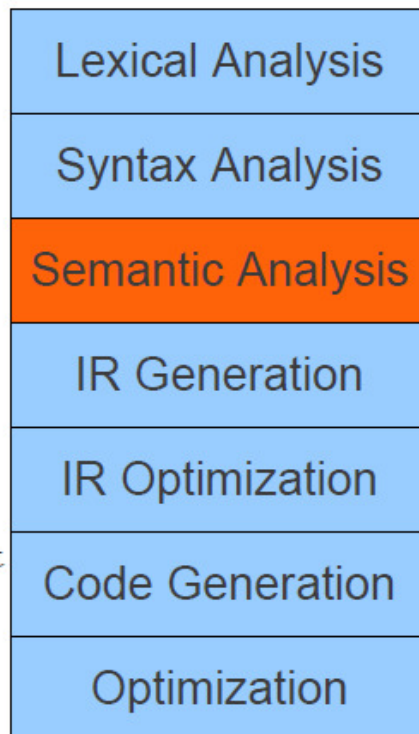
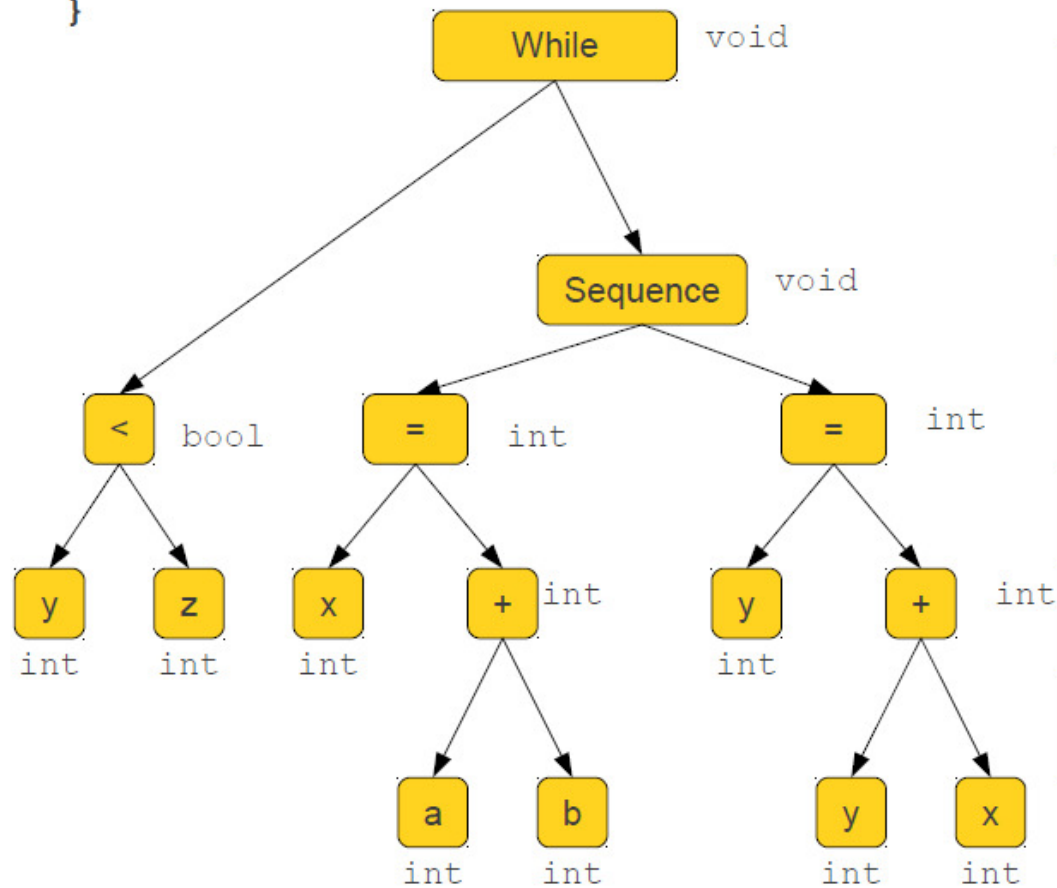
Code Generation

Optimization

```

while (y < z) {
    int x = a + b;
    y += x;
}

```



```
while (y < z) {  
    int x = a + b;  
    y += x;  
}
```

```
Loop: x      = a + b  
      y      = x + y  
      _t1    = y < z  
      if _t1 goto Loop
```

Lexical Analysis

Syntax Analysis

Semantic Analysis

IR Generation

IR Optimization

Code Generation

Optimization

```
while (y < z) {  
    int x = a + b;  
    y += x;  
}
```

```
      x      = a + b  
Loop: y      = x + y  
      _t1    = y < z  
      if _t1 goto Loop
```

Lexical Analysis

Syntax Analysis

Semantic Analysis

IR Generation

IR Optimization

Code Generation

Optimization

```
while (y < z) {  
    int x = a + b;  
    y += x;  
}
```

```
          add $1, $2, $3  
Loop:     add $4, $1, $4  
          slt $6, $1, $5  
          beq $6, loop
```

Lexical Analysis

Syntax Analysis

Semantic Analysis

IR Generation

IR Optimization

Code Generation

Optimization

```
while (y < z) {  
    int x = a + b;  
    y += x;  
}
```

```
      add $1, $2, $3  
Loop: add $4, $1, $4  
      blt $1, $5, loop
```

Lexical Analysis

Syntax Analysis

Semantic Analysis

IR Generation

IR Optimization

Code Generation

Optimization